

Greedy

Greedy చాలా సులభం – మూడు పంక్తులు, sort చేసి ముందుకి నడవడం. కష్టం code lo లేదు. కష్టం **అది సరైనదని నిరూపించడంలో** ఉంది.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది – విరామాల problem కి అయిదు సహజమైన greedy నియమాల్లో **ఒక్కటే** సరైనది; మిగతావి 12.2% నుంచి **76.5%** వరకు తప్పు. రెండోది – నాణేల greedy భారతీయ/అమెరికన్ కరెన్సీకి సరైనది, మరియు **మూడు-నాణెం వ్యవస్థల్లో 62%** కి తప్పు. మూడోది – నా పరీక్ష **వృత్తాకారంగా** ఉంది, మరియు అది ఒక bug ని దాచింది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node lo run` చేసినదే – §8 యొక్క counterexample ని నేను వెతికి కనుగొన్నాను, ఎక్కడి నుంచో తీసుకోలేదు.

GREEDY ఈ SERIES LO ఇతర PATTERNS కి వ్యతిరేకం

Deep 01-06 lo ప్రతి algorithm కీ ఒక ప్రకటించిన ఊహ ఉంది: predicate ఏకదిశ, బరువులు ≥ 0 . ఆ ఊహ మీకు తెలుసు, మరియు దాన్ని తనిఖీ చేయగలరు.

Greedy lo ఊహ ప్రకటించబడదు. "ముందుగా ముగిసేది తీసుకో" అనే నియమం ఎందుకు సరైనదో code చెప్పుదు. అది **మీ తలలో** ఉంది – లేదా లేదు.

కాబట్టి ఇక్కడ ప్రశ్న "ఏ ఊహ విరిగింది" కాదు. ప్రశ్న: "ఈ నియమం సరైనదని మీకు ఎలా తెలుసు?"

§4 lo కొలుస్తాం – అయిదు నియమాలూ సహేతుకంగా అనిపిస్తాయి, మరియు ఒక్కటే సరైనది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Greedy ఎప్పుడు పనిచేస్తుంది
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: ఏ నియమం

3. Step – అయిదు సహజమైన నియమాలు
4. మొదటి విరుపు – అయిదింటిలో ఒక్కటే సరైనది
5. Step – మార్పిడి వాదన (exchange argument)

Part 3 – రెండో విరుపు: నాణేలు

6. Step – పెద్ద నాణెం ముందు
7. Step – counterexample ని వెతకడం
8. రెండో విరుపు – మూడు-నాణెం వ్యవస్థల్లో 62%

Part 4 – మూడో విరుపు: నా పరీక్ష

9. మూడో విరుపు – వృత్తాకార నియమం
10. Step – స్వతంత్ర ఆధారం

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code
12. 27,000 cases · mutation testing

13. Greedy సరైనదైన ఒక problem
14. Follow-ups – matroids, Huffman, scheduling
15. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Greedy ఎప్పుడు పనిచేస్తుంది

Greedy ఒక్క పని చేస్తుంది: **ఇప్పుడు అత్యుత్తమంగా కనిపించేది తీసుకో, వెనక్కి చూడకు.**

అది సరైనది కావాలంటే **రెండు** విషయాలు నిజం కావాలి:

GREEDY యొక్క రెండు షరతులు

1. Greedy choice property – ఏదో ఒక అత్యుత్తమ పరిష్కారంలో **మొదటి greedy ఎంపిక ఉంటుంది.**

2. Optimal substructure – ఆ ఎంపిక తీసేసిన తర్వాత మిగిలేది **అదే problem, చిన్నది.**

రెండోది DP కి కూడా కావాలి (Deep 04). **మొదటిదే greedy ని ప్రత్యేకంగా చేస్తుంది** – మరియు అదే నిరూపించడం కష్టం.

మరియు అది **problem** కి కాదు, **మీరు ఎంచుకున్న నియమానికి** వర్తిస్తుంది. §4 lo అదే చూస్తాం: ఒకే problem, అయిదు నియమాలు, ఒక్కటే ఈ షరతుని పాటిస్తుంది.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
నా greedy నియమం ఏమిటి – ఖచ్చితంగా?	"greedy వాడతాను" ఒక జవాబు కాదు. §4 lo అయిదు వేర్వేరు greedy లు ఉన్నాయి
అది సరైనదని ఎందుకు ?	§5 – ఒక మార్పిడి వాదన రాయగలరా?
Counterexample వెతికారా?	§7 – చిన్న inputs మీద brute force తో పోల్చడం చౌక
వస్తువులని భాగాలుగా తీసుకోవచ్చా?	అవును అంటే greedy నిరూపితంగా సరైనది (§13). కాదంటే NP-hard
Input కి ఒక ప్రత్యేక నిర్మాణం ఉందా?	§8 – కరెన్సీ వ్యవస్థలు greedy కోసం రూపొందించబడ్డాయి

ప్రశ్న	ఎందుకు అడుగుతున్నాం
DP ఎంత ఖరీదు?	Greedy విఫలమైతే అదే ప్రత్యామ్నాయం, మరియు తరచుగా ఖరీదు తక్కువే

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఈ greedy నియమానికి ఒక counterexample వెతికారా?"

ఎందుకంటే అది చౌకైన పని: 5-7 మూలకాల inputs మీద brute force నడిపి greedy తో పోల్చడం. §4 lo నేను 1 లక్ష cases నడిపాను, మరియు నాలుగు నియమాలు వెంటనే విఫలమయ్యాయి.

మరియు §8 lo – అన్ని మూడు-నాణెం వ్యవస్థలని పరీక్షించడానికి 276 ప్రయత్నాలు చాలు.

Greedy ని "నిరూపించడం" కష్టం. దాన్ని "తప్పని చూపించడం" చాలా చౌక.

Part 2 – మొదటి విరుపు: ఏ నియమం

3. Step – అయిదు సహజమైన నియమాలు

Problem: విరామాల జాబితా ఇచ్చారు. ఢీకొట్టకుండా గరిష్ట సంఖ్యలో ఎన్ని ఎంచుకోగలరు?

అయిదు నియమాలు, అన్నీ సహేతుకంగా అనిపిస్తాయి:

'ముందుగా ముగిసేది' : $(x, y) \Rightarrow x[1] - y[1]$
'ముందుగా మొదలయ్యేది' : $(x, y) \Rightarrow x[0] - y[0]$
'అతి తక్కువ నిడివి' : $(x, y) \Rightarrow (x[1]-x[0]) - (y[1]-y[0])$
'అతి పొడవైన నిడివి' : $(x, y) \Rightarrow (y[1]-y[0]) - (x[1]-x[0])$
'అలస్యంగా మొదలయ్యేది' : $(x, y) \Rightarrow y[0] - x[0]$

"అతి తక్కువ నిడివి" చాలా నమ్మకంగా అనిపిస్తుంది – చిన్న విరామాలు తక్కువ చోటు తీసుకుంటాయి కదా?

4. మొదటి విరుపు – అయిదింటిలో ఒక్కటే సరైనది

గరిష్ట ఢీకొట్టని విరామాలు · 1,00,000 యాదృచ్ఛిక cases · brute force తో పోల్చినవి

greedy నియమం	తప్పు జవాబులు	%
ముందుగా ముగిసేది (end)	0	0.0%
ముందుగా మొదలయ్యేది (start)	12,153	12.2%
అతి తక్కువ నిడివి	47,284	47.3%
అతి పొడవైన నిడివి	62,363	62.4%
అలస్యంగా మొదలయ్యేది	76,468	76.5%

"అతి తక్కువ నిడివి" – 47.3% తప్పు

[0,6] [3,9] [9,12] [8,9] → greedy 2, నిజం 3

అతి చిన్న విరామం [8,9] . దాన్ని తీసుకుంటే [3,9] మరియు [9,12] లో ఒకటి పోతుంది.

సరైన జవాబు: [0,6] [8,9] [9,12] – మూడు. కానీ greedy [8,9] తర్వాత [9,12] తీసుకుని ఆగిపోతుంది, ఎందుకంటే [0,6] ని అప్పటికే పరిశీలించి వదిలేసింది.

"చిన్నది తక్కువ చోటు తీసుకుంటుంది" అనేది నిజం. కానీ అది ఎక్కడ ఉంది అనేది ముఖ్యం, ఎంతకాదు.

"ముందుగా మొదలయ్యేది" - 12.2% తప్పు, మరియు అదే అత్యంత ప్రమాదకరమైనది

12.2% అంటే 88% సార్లు సరైనది. మీ ఉదాహరణలు నెగ్గుతాయి, మీ intuition ధృవీకరించబడుతుంది, మరియు ఒక పెద్ద test దగ్గర విఫలం.

[8,13] [11,16] [0,1] [6,12] [4,8] [3,6] [6,10] → greedy 3, నిజం 4.

Deep 01 lo 83%, Deep 03 lo 99.3%, Deep 05 lo 98.8% - ఇప్పుడు 88%. ఈ series lo ఇదే కేంద్ర నమూనా.

5. Step – మార్పిడి వాదన (exchange argument)

"ముందుగా ముగిసేది" ఎందుకు సరైనది? దానికి ఒక నిరూపణ ఉంది, మరియు అది ఒక పేరా:

మార్పిడి వాదన – GREEDY నిరూపణల యొక్క ప్రామాణిక ఆకారం

ఏదైనా ఒక అత్యుత్తమ పరిష్కారం OPT తీసుకోండి. దాని మొదటి విరామం x , మరియు greedy ఎంచుకున్నది g (అన్నిటికంటే ముందుగా ముగిసేది).

నిర్వచనం ప్రకారం g ముగింపు $\leq x$ ముగింపు.

కాబట్టి OPT lo x ని తీసేసి g పెట్టొచ్చు - g ముందే ముగుస్తుంది కాబట్టి తర్వాతి విరామాలేవీ ఢీకొట్టవు. పరిమాణం మారలేదు, కాబట్టి ఇది కూడా అత్యుత్తమమే.

అంటే greedy ఎంపికతో మొదలయ్యే ఒక అత్యుత్తమ పరిష్కారం ఉంది - అదే §1 యొక్క షరతు 1. ఆపై మిగిలిన problem చిన్నది, మరియు అదే వాదన మళ్ళీ వర్తిస్తుంది.

ఇప్పుడు అదే వాదనని "అతి తక్కువ నిడివి" కి రాయడానికి ప్రయత్నించండి

g అతి చిన్న విరామం. OPT lo x ని g తో మార్చాలంటే - అది సాధ్యం కాదు, ఎందుకంటే g మధ్యలో ఎక్కడైనా ఉండొచ్చు, మరియు అది OPT lo ఉన్న రెండు విరామాలని ఢీకొట్టొచ్చు.

వాదన ఇక్కడే ఆగిపోతుంది. మరియు అది ఆగిపోవడమే సంకేతం.

Deep 01 lo "pred(x) నిజమైతే pred(x+1) నిజం, ఎందుకంటే ____" • Deep 02 lo "I వెనక్కి రావాల్సిన అవసరం లేదు, ఎందుకంటే ____" - ఇక్కడ "greedy ఎంపికని OPT lo పెట్టొచ్చు, ఎందుకంటే ____". మూడూ ఒకే పరీక్ష: ఖాళీని పూరించలేకపోతే, ఆగండి.

Part 3 – రెండో విరుపు: నాణేలు

6. Step – పెద్ద నాణెం ముందు

```
function coinChangeGreedy(coins, amount) {  
  const s = [...coins].sort((a, b) => b - a); // పెద్దవి ముందు  
  let n = 0, rem = amount;  
  for (const c of s) { n += Math.floor(rem / c); rem %= c; }  
  return rem === 0 ? n : -1;  
}
```

ఇది మనం **నిజ జీవితంలో** చేసేదే. ₹87 ఇవ్వాలంటే: ₹50, ₹20, ₹10, ₹5, ₹2 – పెద్దవి ముందు.

మరియు అది **పనిచేస్తుంది**.

7. Step – counterexample ని వెతకడం

"పనిచేస్తుంది" అనేది ఒక పరిశీలన, నిరూపణ కాదు. కాబట్టి వెతికాను – **అన్ని** మూడు-నాణెం వ్యవస్థలని, DP తో పోల్చి.

```
function smallestFail(coins, limit = 200) {  
  for (let a = 1; a <= limit; a++)  
    if (greedy(coins, a) !== optimal(coins, a)) return { amt: a, ... };  
  return null; // ఈ పరిధిలో విఫలం కాలేదు  
}
```

8. రెండో విరుపు – మూడు-నాణెం వ్యవస్థల్లో 62%

బాగా తెలిసిన నాణెం వ్యవస్థలు

నాణేలు	greedy సరైనదా	విఫలమయ్యే అతి చిన్న మొత్తం
[1, 2, 5, 10, 20, 50, 100]		అవును ✓ –
[1, 5, 10, 25]		అవును ✓ –
[1, 3, 4]		కాదు 6 → greedy 3 నాణేలు, నిజం 2
[1, 5, 10, 20, 25]		కాదు 40 → greedy 3 నాణేలు, నిజం 2
[1, 7, 10]		కాదు 14 → greedy 5 నాణేలు, నిజం 2

అతి చిన్న COUNTEREXAMPLE - [1, 3, 4] · మొత్తం 6

greedy: $4 + 1 + 1 = 3$ నాణేలు

నిజం: $3 + 3 = 2$ నాణేలు

మూడు నాణేలు, ఒక అంక మొత్తం. ఇది ఒక అరుదైన, కృత్రిమ సందర్భం కాదు – ఇది **అత్యంత చిన్న** సందర్భం.

మరియు నాలుగో వరుస చూడండి: [1, 5, 10, 20, 25] – అమెరికన్ కరెన్సీకి **ఒక ₹20 నాణెం** కలిపితే చాలు. 40 కి greedy $25+10+5$ (3 నాణేలు), నిజం $20+20$ (2).

మూడు నాణేల వ్యవస్థలన్నీ ($1 < b < c \leq 25$) · greedy ఎన్నింటిలో విఫలం?

పరీక్షించిన వ్యవస్థలు : 276

greedy విఫలమైనవి : 170 (62%)

62% – అంటే GREEDY పనిచేయడం నియమం కాదు, మినహాయింపు

మీరు యాదృచ్ఛికంగా ఒక నాణెం వ్యవస్థ ఎంచుకుంటే, greedy **విఫలమయ్యే** అవకాశమే ఎక్కువ.

మరి భారతీయ/అమెరికన్ కరెన్సీకి ఎందుకు పనిచేస్తుంది?

ఎందుకంటే అవి అలా రూపొందించబడ్డాయి. కరెన్సీ వ్యవస్థలు వందల ఏళ్ళుగా "మనుషులు సులభంగా చిల్లర ఇవ్వగలగాలి" అనే లక్ష్యంతో పరిణామం చెందాయి – మరియు "సులభంగా" అంటే సరిగ్గా "greedy పనిచేస్తుంది".

మీ intuition తప్పు కాదు – అది ఒక రూపొందించబడిన ప్రత్యేక సందర్భం నుంచి వచ్చింది. ఆ ప్రత్యేకతని మీరు గమనించకపోవడమే సమస్య.

Part 4 – మూడో విరుపు: నా పరీక్ష

9. మూడో విరుపు – వ్యత్యాస నియమం

§12 కోసం fuzz రాసినప్పుడు నేను ఒక నియమం పెట్టాను:

```
const canon = M.isCanonical(uniq, 40);  
if (canon && g !== o) bad.push(`canonical అన్నాం కానీ greedy ≠ optimal`);
```

సహజంగా అనిపిస్తుంది: వ్యవస్థ canonical అయితే greedy = optimal కావాలి.

అపై `coinChangeGreedy` ని చిన్న నాణేలు ముందు (స్పష్టంగా తప్పు) అని మార్చి పరీక్షించాను:

greedy 10 పెద్దవి ముందు కి బదులు చిన్నవి ముందు → 0/120 విఫలం

స్పష్టంగా విరిగిన **GREEDY** - ఒక్క నియమమూ విఫలం కాలేదు

కారణం: `isCanonical` తనే `coinChangeGreedy` ని వాడుతుంది.

Greedy ని మార్చినప్పుడు `isCanonical` కూడా మారింది - ఇప్పుడు అది ఏ వ్యవస్థనీ **canonical** అనదు.

కాబట్టి `if (canon && ...)` అనే శాఖ ఎప్పుడూ నడవలేదు.

నా నియమం greedy ని greedy తోనే పోల్చుతోంది. అది ఎప్పుడూ ఏకీభవిస్తుంది.

10. Step – స్వతంత్ర ఆధారం

పరిష్కారం: పరీక్షించబడుతున్న code మీద ఆధారపడని ఒక ఆధారం.

```
// *తెలిసిన* canonical వ్యవస్థల మీద greedy తప్పక optimal కావాలి.  
// (ఇది isCanonical మీద ఆధారపడదు - లేకపోతే పరీక్ష వ్యత్యాసం అవుతుంది.)  
for (const known of [[1,5,10,25], [1,2,5,10,20,50], [1,10,100]]) {  
  const kg = M.coinChangeGreedy(known, amt), ko = M.coinChangeOptimal(known, amt);  
  if (kg !== ko) bad.push(`తెలిసిన canonical [${known}]: greedy ${kg} ≠ optimal  
  ${ko}`);  
}
```

ఆ మార్పుతో - **120/120**, మరియు ఉదాహరణ `[1,5,10,25] amount 14: greedy 14 ≠ optimal 5`.

ఇది ఈ SERIES LO ఒక కొత్త రకం వైఫల్యం

ఇప్పటిదాకా బతికిన mutations కి నాలుగు కారణాలు చూశాం: బలహీన పరీక్ష, చచ్చిన code, harness పరిమితి, సరిపోని data.

ఇది అయిదోది: పరీక్ష తనని తానే ధృవీకరించుకోవడం.

మరియు ఇది కనిపెట్టడం కష్టం, ఎందుకంటే నియమం చదవడానికి సరైనదే. తప్పు తర్కంలో కాదు – ఆధారం ఎక్కడి నుంచి వస్తోంది అనేదాంట్లో.

ఒక పరీక్ష ఎప్పుడూ అది పరీక్షిస్తున్న దాని *బయట* నుంచి సత్యాన్ని తీసుకోవాలి – brute force, తెలిసిన జవాబులు, లేదా ఒక వేరే algorithm (Deep 05 lo Floyd-Warshall).

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code

```
// గరిష్ట ఢీకొట్టని విరామాలు. సరైన నియమం: **ముందుగా ముగిసేది** (§5 lo నిరూపణ).
function activitySelection(intervals) {
  const s = [...intervals].sort((a, b) => a[1] - b[1]);
  const out = [];
  let end = -Infinity;
  for (const iv of s) if (iv[0] >= end) { out.push(iv); end = iv[1]; }
  return out;
}
// ఈ వ్యవస్థకి greedy సరైనదేనా? – limit వరకు పరీక్షించడం
function isCanonical(coins, limit = 200) {
  if (!coins.includes(1)) return false;
  for (let a = 1; a <= limit; a++)
    if (coinChangeGreedy(coins, a) !== coinChangeOptimal(coins, a)) return false;
  return true;
}
```

iv[0] >= end lo >= ముఖ్యం – ఒకటి ముగిసిన సరిగ్గా అదే క్షణంలో ఇంకొకటి మొదలవడం చెల్లుతుంది.
> గా మార్చితే 120/120 విఫలం.

12. 27,000 cases • mutation testing

తొమ్మిది నియమాలు: **activitySelection = brute force**, ఎంచుకున్నవి ఢీకొట్టవు, అన్న అసలు జాబితాలోనివే, **greedy ఎప్పుడూ optimal** కంటే తక్కువ నాణేలు ఇవ్వదు, 1 ఉంటే ప్రతి మొత్తమూ సాధ్యం, 1 లేకపోతే **canonical కాదు**, తెలిసిన **canonical వ్యవస్థల మీద greedy = optimal**, **coinChangeOptimal = BFS**, భిన్న **knapsack = brute force**.

300 runs • 12,000 interval cases • 9,000 coin cases • 6,000 knapsacks
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

intervals	12,000
coins	9,000
canonical	5,426
nonCanonical	3,574
frac	6,000

==== mutation testing ====

మార్పు లేని code → 0/300 విఫలం
 విరామాలని ముగింపుకి బదులు *మొదలు* ప్రకారం sort → 120/120 విఫలం
 ఉదా: activitySelection = 3, ఆశించినది 4
 విరామాలని *నిడివి* ప్రకారం sort → 120/120 విఫలం
 ఉదా: activitySelection = 2, ఆశించినది 3
 `iv[0] >= end` ని `iv[0] > end` గా → 120/120 విఫలం
 greedy lo పెద్దవి ముందు కి బదులు చిన్నవి ముందు → 120/120 విఫలం
 ఉదా: తెలిసిన canonical [1,5,10,25] amount 14: greedy 14 ≠ optimal 5
 coinChangeOptimal lo d[0] = 0 తీసేస్తే → 120/120 విఫలం
 isCanonical lo 1 ఉందా అనే తనిఖీ తీసేస్తే → 117/120 విఫలం
 ఉదా: [8] lo 1 లేదు, అయినా canonical అన్నాం
 fractional knapsack ని విలువ/బరువు కి బదులు విలువ ప్రకారం → 120/120 విఫలం
 ఉదా: fractionalKnapsack = 25, ఆశించినది 27.5

ఏడు మార్పులూ పట్టుబడ్డాయి – **కానీ రెండు మాత్రమే మొదటి ప్రయత్నంలో**. ఒకటికి ₹10 యొక్క స్వతంత్ర ఆధారం కావాల్సి వచ్చింది; ఇంకొకదానికి **1 లేని నాణెం వ్యవస్థలు** (నా data lo అవి లేవు – ఈ series lo అయిదేసారి అదే పాఠం).

13. Greedy సరైనదైన ఒక problem

చివరి mutation ఒక ముఖ్యమైన విరుద్ధతని చూపిస్తుంది: **భిన్న knapsack** (fractional knapsack) lo greedy **నిరూపితంగా** సరైనది.

```
const s = [...items].sort((a, b) => b.v / b.w - a.v / a.w); // విలువ/బరువు ప్రకారం
```

0/1 KNAPSACK NP-HARD · భిన్న KNAPSACK GREEDY తో $O(N \log N)$

తేడా ఒక్కటే: **వస్తువుని ముక్కలు చేయొచ్చా**.

చేయొచ్చు అయితే – అత్యధిక విలువ/బరువు నిపుణ్ణి ఉన్నదాన్ని **నిండేవరకు** తీసుకోవడం ఎప్పుడూ సరైనది. మార్పిడి వాదన సులభం: ఏదైనా పరిష్కారంలో తక్కువ నిపుణ్ణి ఉన్న ముక్కని ఎక్కువ నిపుణ్ణి ఉన్నదానితో మార్పితే విలువ **తగ్గదు**.

చేయలేకపోతే – ఆ మార్పిడి **సాధ్యం కాదు** (సగం వస్తువు తీసుకోలేరు), వాదన ఆగిపోతుంది, మరియు problem NP-hard అవుతుంది.

ఒక్క పదం – "భిన్నం" – ఒక $O(n \log n)$ algorithm మరియు ఒక NP-hard problem మధ్య తేడా.

మరియు నిపుణ్ణి కాకుండా **విలువ ప్రకారం sort చేస్తే: 25 కి బదులు 27.5, 120/120 విఫలం.**

14. Follow-ups – matroids, Huffman, scheduling

"Greedy ఎప్పుడు పనిచేస్తుందో ఒక సాధారణ సిద్ధాంతం ఉందా?"

అవును – **matroid theory**. మీ problem ఒక matroid నిర్మాణాన్ని పొటిస్తే, greedy నిరూపితంగా అత్యుత్తమం. (Minimum spanning tree – Kruskal – దీనికి ఒక ఉదాహరణ.) కానీ చాలా interview problems matroids కావు, మరియు ఆ సిద్ధాంతాన్ని వర్తింపజేయడం కంటే **§5 యొక్క మార్పిడి వాదన రాయడం** చాలా వేగం. **నేను matroids ని కొలవలేదు.**

"Huffman coding ఒక greedy – అది ఎందుకు సరైనది?"

అదే మార్పిడి వాదన: అత్యంత అరుదైన రెండు అక్షరాలు అత్యుత్తమ వృక్షంలో **అత్యంత లోతులో, తేబుట్టువులుగా** ఉంటాయి. కాబట్టి వాటిని ముందు కలపడం ఎప్పుడూ సురక్షితం. ఇది §5 యొక్క నిర్మాణమే – "greedy ఎంపికతో మొదలయ్యే ఒక అత్యుత్తమ పరిష్కారం ఉంది".

"Scheduling problems లో ఏ నియమం?"

అది **ఏమి optimise చేస్తున్నారో** దాని మీద ఆధారపడుతుంది, మరియు నియమం ప్రతిసారీ వేరు:

లక్ష్యం	సరైన greedy నియమం
గరిష్ట సంఖ్యలో పనులు	ముందుగా ముగిసేది (§4)
సగటు నిరీక్షణ కనిష్టం	అతి తక్కువ నిడివి ముందు
గడువులు మించడం కనిష్టం	ముందుగా గడువు ఉన్నది

అదే మూడు నియమాలూ §4 లో ఉన్నాయి, మరియు అక్కడ రెండు తప్పు. **నియమం problem ని బట్టి మారుతుంది, మరియు అదే greedy ని ప్రమాదకరంగా చేస్తుంది.**

"ఈ doc లో నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **matroids, Huffman**, మరియు §8 యొక్క శోధనని **మూడు నాణేలకి మించి** – నాలుగు-నాణెం వ్యవస్థల్లో ఎన్ని శాతం విఫలమో నేను పరీక్షించలేదు.

15. ఏమి నేర్చుకున్నాం

- "Greedy వాడతాను" ఒక జవాబు కాదు.** ఒకే problem కి అయిదు greedy నియమాలు ఉన్నాయి, మరియు ఒక్కటే సరైనది.
- తప్పు నియమాలు సహేతుకంగా అనిపిస్తాయి.** "అతి తక్కువ నిడివి" – **47.3% తప్పు**. "ముందుగా మొదలయ్యేది" – **12.2%**, అంటే 88% సార్లు సరైనది.
- మార్పిడి వాదన ఒక పరీక్ష, ఒక ఆచారం కాదు.** "greedy ఎంపికని OPT లో పెట్టొచ్చు, ఎందుకంటే ____" – పూరించలేకపోతే ఆ నియమం తప్పు.
- Greedy ని నిరూపించడం కష్టం; తప్పుని చూపించడం చౌక.** 5 మూలకాల inputs మీద brute force చాలు.
- అతి చిన్న coin counterexample: [1, 3, 4] మొత్తం 6.** greedy 3 నాణేలు, నిజం 2.

6. Greedy పనిచేయడం నియమం కాదు, మినహాయింపు. మూడు-నాణెం వ్యవస్థల్లో 62% విఫలం. కరెన్సీలు greedy కోసం రూపొందించబడ్డాయి.

7. ఒక్క పదం ఒక NP-hard problem ని $O(n \log n)$ చేస్తుంది. "భిన్నం" – మరియు అది మార్పిడి వాదనని సాధ్యం చేయడం వల్లే.

8. ఒక పరీక్ష తనని తానే ధృవీకరించుకోకూడదు. `isCanonical` greedy ని వాడుతుంది, కాబట్టి "canonical $\Rightarrow$ greedy = optimal" అనే నియమం వ్యత్యాసం – స్పష్టంగా విరిగిన greedy ని 0/120.

9. సత్యం ఎప్పుడూ బయటి నుంచి రావాలి. Brute force, తెలిసిన జవాబులు, లేదా ఒక వేరే algorithm.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Greedy ఒక algorithm కాదు – అది ఒక వాదన, మరియు ఆ వాదనని మీరు రాయాలి.

Code మూడు పంక్తులు, మరియు అవి ఎప్పుడూ ఒక జవాబు ఇస్తాయి. అది అత్యుత్తమ జవాబా అనేది code lo ఎక్కడా లేదు.

కాబట్టి greedy రాసే ముందు §5 యొక్క వాక్యాన్ని పూర్తి చేయండి. చేయలేకపోతే – DP రాయండి, ఇది కొంచెం నెమ్మది కావచ్చు, మరియు అది నిజంగా సరైనది.

మరియు ఎప్పుడూ counterexample వెతకండి. §8 lo అది 276 ప్రయత్నాలు తీసుకుంది.